

CAPSTONE PROJECT REPORT

Report 2 – Project Management Plan

Table of Contents

I. Record of Changes.....	3
II. Project Management Plan.....	4
1. Overview.....	4
1.1 Scope & Estimation.....	4
1.2 Project Objectives.....	10
1.3 Project Risks.....	10
2. Management Approach.....	11
2.1 Project Process.....	12
2.2 Quality Management.....	15
2.3 Training Plan.....	17
3. Project Deliverables.....	18
4. Responsibility Assignments.....	19
5. Project Communications.....	22
6. Configuration Management.....	23
6.1 Document Management.....	23
6.2 Source Code Management.....	25
6.3 Tools & Infrastructures.....	29

I. Record of Changes

Date	A* M, D	In charge	Change Description
14/09/2026	A	Le Thinh	1. Overview
14/09/2026	A	Doan Thi Yen Nhi	2. Management Approach
15/09/2026	A	Nguyen Chi Duong	3. Project Deliverables
16/09/2026	A	Nguyen Chi Duong	4. Responsibility Assignments
17/09/2026	A	Le Dang Khoa	5. Project Communications
18/09/2026	A	Pham Van Hoai	6. Configuration Management

*A - Added M - Modified D - Deleted

II. Project Management Plan

1. Overview

1.1 Scope & Estimation

#	WBS Item	Complexity	Est. Effort (man-days)
1	Reports		81
1.1	Report 1	Simple	4
1.2	Report 2	Medium	7
1.3	Report 3	Complex	21
1.4	Report 4	Complex	21
1.5	Report 5	Complex	14
1.6	Report 6	Medium	7
1.7	Report 7	Medium	7
2	Feature 1: Authentication & Account Management		13
2.1	Initialize User Account	Medium	2
2.2	Login	Medium	2
2.3	Logout	Simple	1
2.4	Forgot / Reset Password	Simple	2
2.5	Change Password	Simple	1
2.6	Multi-Factor Authentication (MFA)	Simple	3
2.7	Lock / Unlock User Account	Simple	1
2.8	View Login History	Medium	1

3	Feature 2: User Management & Authorization		14
3.1	Add User	Medium	1
3.2	Edit User Information	Medium	1
3.3	Deactivate / Delete User	Medium	1
3.4	Assign User Role	Medium	1
3.5	Create Custom Role	Medium	2
3.6	Configure Detailed Permissions	Complex	4
3.7	View Users by Department	Complex	1
3.8	Import Users from Excel	Simple	3
4	Feature 3: IT Asset Management		21
4.1	Add IT Asset	Medium	2
4.2	Edit IT Asset	Medium	1
4.3	Delete / Stop Tracking Asset	Medium	1
4.4	View Asset List	Simple	1
4.5	Search and Filter Assets	Medium	2
4.6	Classify Asset Criticality	Medium	2
4.7	Assign Asset Owner	Medium	1
4.8	Import Assets from Excel	Medium	4
4.9	Export Asset List	Medium	3
4.10	View Asset Change History	Medium	4

5	Feature 4: Risk Assessment		28
5.1	Create Risk Assessment	Medium	2
5.2	Identify Threat	Medium	2
5.3	Identify Vulnerability	Medium	2
5.4	Automatically Calculate Risk Score	Medium	3
5.5	Suggest Risk Treatment Measures	Medium	3
5.6	Submit Risk Treatment Plan for Approval	Medium	2
5.7	Approve Risk Treatment Plan	Medium	2
5.8	Reject / Request Risk Assessment Revision	Medium	2
5.9	Track Risk Treatment Progress	Medium	2
5.10	Close Resolved Risk	Medium	1
5.11	View Risk Heat Map	Medium	3
5.12	Compare Risk Changes Between Assessment Periods	Medium	4
6	Feature 5: Policy & Compliance Control		22
6.1	Draft Information Security Policy	Medium	2
6.2	Publish Official Policy Version	Medium	2
6.3	Update Policy / Create New Version	Medium	2
6.4	Assign Policy to Department	Medium	2
6.5	Confirm Policy Reading and Understanding	Medium	1
6.6	Map Controls to Standard Framework	Medium	4

6.7	Assess Control Compliance	Medium	3
6.8	Upload Compliance Evidence	Medium	2
6.9	View Policy Version History	Simple	2
6.10	Receive Compliance Reminders	Simple	2
7	Feature 6: Information Security Incident Management		20
7.1	Report New Incident	Medium	2
7.2	Classify Incident Severity	Medium	2
7.3	Assign Incident Handler	Medium	1
7.4	Update Incident Handling Progress	Medium	1
7.5	Attach Incident Evidence and Logs	Medium	2
7.6	Escalate Critical Incident	Medium	2
7.7	Close Resolved Incident	Medium	1
7.8	Create Post-Incident Report	Medium	4
7.9	View Incident History	Medium	2
7.10	Link Incident to Related Risk	Medium	3
8	Feature 7: AI Anomaly Detection & Alerts		38
8.1	Configure Log / Data Source	Complex	4
8.2	Fine-Tune Anomaly Detection Model	Complex	5
8.3	View Real-Time AI Alerts	Complex	4
8.4	Evaluate Alert Reliability	Medium	4

8.5	Confirm AI Alert as Real Incident	Medium	2
8.6	Mark False Positive	Medium	4
8.7	View AI-Suggested Risk Level	Medium	4
8.8	View Explainable AI Decision	Complex	4
8.9	Automatically Create Incident Draft	Complex	4
8.10	Set Custom Alert Threshold	Medium	3
9	Feature 8: Security Awareness Training		14
9.1	Create Security Awareness Course	Medium	2
9.2	Assign Training Course	Medium	2
9.3	Take Post-Training Assessment	Medium	2
9.4	Track Training Completion	Medium	2
9.5	Send Training Deadline Reminder	Medium	1
9.6	Issue Training Completion Certificate	Medium	2
9.7	View Department Training Completion Report	Medium	3
10	Feature 9: Reports & Dashboard		19
10.1	View Real-Time Risk / Incident Dashboard	Medium	3
10.2	Export Periodic Risk Report	Medium	3
10.3	Export Compliance Report	Medium	3
10.4	Customize Dashboard Widgets	Medium	3
10.5	Compare Security Metrics Between Reporting Periods	Medium	2

10.6	Create Information Security KPI Report	Medium	2
10.7	View Executive Summary Report	Medium	1
10.8	Schedule Automatic Email Reports	Medium	2
11	Feature 10: Notification & System Logs		11
11.1	Send In-System Notification	Medium	1
11.2	Send Email Notification	Simple	2
11.3	Configure Personal Notification Channel	Medium	2
11.4	View User Activity Audit Log	Medium	2
11.5	Search Logs with Advanced Filters	Medium	2
11.6	Export Investigation Logs	Medium	1
11.7	View System Configuration Change History	Medium	1
12	Feature 11: Approval & Workflow		11
12.1	Configure Custom Approval Workflow	Medium	3
12.2	Approve / Reject Request	Medium	2
12.3	Delegate Temporary Approval Authority	Medium	2
12.4	Track Pending Request Status	Medium	1
12.5	Receive Overdue Approval Reminder	Medium	1
12.6	View Approval Decision History	Medium	2
13	Feature 12: Integration & API		8
13.1	Connect Third-Party SIEM / Firewall API	Medium	3

13.2	Schedule Automatic Log Synchronization	Medium	2
13.3	Manage Integration API Keys	Medium	1
13.4	Monitor Integration Connection Status	Medium	1
13.5	View Data Synchronization Error Logs	Medium	1
Total Estimated Effort (man-days)			219

1.2 Project Objectives

#	Testing Stage	Test Coverage	No. of Defects	% of Defect	Notes
1	Reviewing	100% of requirements and design documents	≤ 5	≤ 5%	Review requirements, design, and implementation before proceeding to the next stage
2	Unit Test	≥ 80% of core implemented functions	≤ 15	≤ 10%	Test individual functions/modules during development
3	System Test	≥ 90% of implemented use cases	≤ 8	≤ 5%	Verify integration and end-to-end system behavior
4	Acceptance Test	100% of critical business workflows	≤ 3	≤ 3%	No Critical defect should remain unresolved before final delivery

Milestone Timeliness (%): ≥ 90%

Allocated Effort (man-days): 300

1.3 Project Risks

#	Risk Description	Impact	Possibility	Response Plans
1	A team member leaves the project or is unable to continue participating	High	Low	Clearly assign responsibilities, back up source code and documents on shared systems. Team members should understand the important parts of each

				module so they can provide support or take over tasks when necessary.
2	A team member's computer is damaged or unavailable for an extended period	High	Medium	Use Git to back up source code and cloud storage to back up project documents. Prepare a development environment that can be set up on another device if necessary.
3	The large scope of 101 Use Cases causes the workload to exceed the team's capacity within the 13-week schedule	High	High	Clearly define and control the project scope from the beginning, prioritize important functions, monitor progress by module, and avoid adding functions outside the agreed scope.
4	Inaccurate effort estimation causes some functions to take longer than expected	High	High	Track actual effort weekly, identify tasks at risk of delay early, and redistribute tasks among team members when necessary.
5	The AI Anomaly Detection module is difficult to implement or does not meet the expected requirements	High	Medium	Develop an AI prototype early, divide the module into smaller tasks, and test it with sample data. If the initial approach is too complex, switch to a simpler approach that still satisfies the defined scope.
6	Insufficient time remains for integration, testing, and bug fixing before the deadline	High	Medium	Perform integration and testing alongside development instead of postponing testing until the end. Track defects continuously and reserve sufficient time before Week 13 for system and acceptance testing.

2. Management Approach

The SecuraAI project will apply an Agile development approach based on the Scrum framework to manage and implement the project. The Agile approach allows the team to develop the system incrementally, receive feedback regularly, identify problems early, and adapt to changes during development.

This approach is suitable for SecuraAI because the project has a relatively large scope with 101 Use Cases, multiple functional modules, and an AI Anomaly Detection module that may require

continuous testing and improvement. The project also has a 13-week schedule, so the team needs to continuously monitor progress and quality instead of waiting until the end of the project.

2.1 Project Process

2.1.1 Agile Scrum Development Model

Agile is a software development approach that focuses on iterative and incremental development, continuous customer feedback, collaboration among team members, and responding to changes.

The team will apply the Scrum framework, in which the project is divided into several short development iterations called Sprints. At the beginning of each Sprint, the team selects a group of prioritized requirements and plans the tasks to be completed. During the Sprint, team members develop, integrate, and test the selected functions. At the end of the Sprint, the completed functions are reviewed and the team evaluates the results before starting the next Sprint.

The basic Scrum process can be represented as:

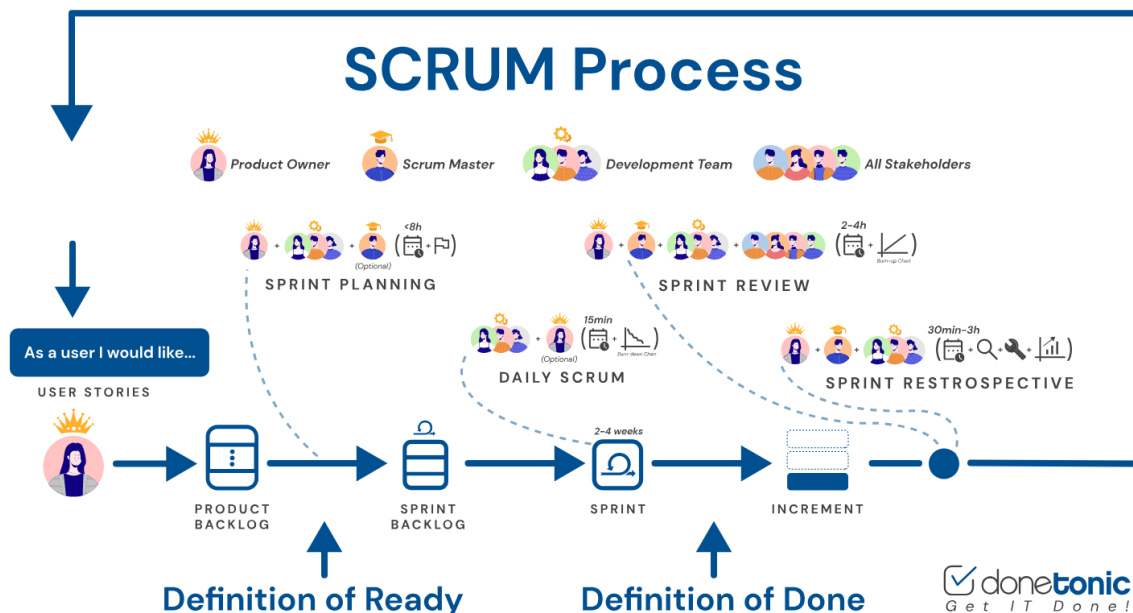


Figure 1: The basic Scrum process¹

2.1.2 Scrum Process Applied to SecuraAI

For the SecuraAI project, the team will organize the development process into several Sprints throughout the 13-week project schedule.

1. Product Backlog

The project requirements and Use Cases are maintained in the Product Backlog. The team will prioritize the functions based on business importance, dependencies, complexity, and project risks.

Examples of prioritized functions include:

- Authentication and Account Management

¹ <https://donetonic.com/scrum-methodology/>

- User Management and Authorization
- IT Asset Management
- Risk Assessment
- Policy and Compliance Control
- Information Security Incident Management
- AI Anomaly Detection and Alerts
- Security Awareness Training
- Reports and Dashboard
- Notification and System Logs
- Approval and Workflow
- Integration and API
- These functions are defined in the project scope and estimation.

2. Sprint Planning

At the beginning of each Sprint, the team reviews the Product Backlog and selects the Use Cases that can realistically be completed within the Sprint.

Tasks are assigned to team members according to their responsibilities and technical skills. The team also identifies dependencies, risks, and expected deliverables for the Sprint.

3. Development and Testing

During the Sprint, developers implement the selected functions. Testing is performed continuously during development instead of being postponed until the end of the project.

The team will perform activities such as:

- Coding and code review
- Unit testing
- Integration testing
- Database testing
- Fixing identified defects
- Updating project documents
- Integrating completed modules

For the AI Anomaly Detection module, the team will develop an early prototype, test it with sample data, and improve the model based on testing results. This is consistent with the project's risk response for the AI module.

4. Sprint Review

At the end of each Sprint, the team reviews the completed functions and checks whether they satisfy the defined requirements and acceptance criteria.

Defects or incomplete functions are recorded and added to the appropriate backlog for further improvement.

5. Sprint Retrospective

After each Sprint, the team discusses what went well, what problems occurred, and what should be improved in the next Sprint.

Possible improvement actions include:

- Redistributing tasks when workload is unbalanced
- Improving communication between team members
- Improving coding and testing practices
- Updating development priorities
- Identifying risks earlier

This is particularly critical as inaccurate effort estimation and insufficient testing time are identified as key project risks

6. Increment

At the end of each Sprint, the team should have a working increment of the SecuraAI system. Completed functions are integrated with existing functions so that the system gradually becomes more complete.

2.1.3 Advantages of Agile Scrum

The Scrum approach provides several advantages for the SecuraAI project:

- Early feedback: Problems can be identified after each Sprint instead of waiting until the end.
- Continuous testing: Testing is performed together with development, reducing the risk of late defects.
- Better progress monitoring: The team can measure progress after every Sprint.
- Flexibility: The team can adjust priorities when necessary while maintaining the agreed project scope.
- Risk reduction: High-risk functions, especially AI-related functions, can be developed and tested early.
- Team collaboration: Regular meetings and reviews improve communication among team members.
- Incremental delivery: The system is developed module by module instead of implementing the entire system at once.

2.1.4 Disadvantages of Agile Scrum

Although Agile Scrum is suitable for the project, it also has some limitations:

- Frequent changes may make project management more difficult if the scope is not controlled.
- Scrum requires active communication and cooperation among team members.
- Poor estimation may cause some Sprint tasks to remain incomplete.
- The large number of Use Cases may make prioritization and Sprint planning difficult.
- Continuous meetings and reviews require additional time from team members.

To reduce these disadvantages, the team will clearly define the project scope, prioritize important functions, monitor progress regularly, and avoid adding functions outside the agreed scope. This approach also addresses the project's risk of having a large workload caused by the 101 Use Cases.

2.1.5 Why Scrum is Selected

The team selects Scrum because it is appropriate for the characteristics of the SecuraAI project. The project contains many Use Cases and several independent but interconnected modules. In addition,

the AI Anomaly Detection module has a higher level of technical uncertainty and may require several iterations to achieve the expected results.

Using Scrum allows the team to divide the project into manageable Sprints, develop important functions first, continuously test the system, and improve the product based on Sprint results.

Furthermore, the project plan already emphasizes continuous testing, defect tracking, and reserving sufficient time for system and acceptance testing before Week 13.

Therefore, Agile Scrum provides a practical approach for controlling the project's scope, schedule, quality, and technical risks.

2.2 Quality Management

The objective of quality management is to ensure that SecuraAI satisfies the defined functional requirements, quality objectives, and acceptance criteria. The team will combine defect prevention, reviewing, unit testing, system testing, and acceptance testing throughout the development process.

The project quality targets include:

Testing Stage	Target	Defect Limit
Reviewing	100% of requirements and design documents reviewed	≤ 5 defects
Unit Test	$\geq 80\%$ of core implemented functions	≤ 15 defects
System Test	$\geq 90\%$ of implemented Use Cases	≤ 8 defects
Acceptance Test	100% of critical business workflows	≤ 3 defects

The project also targets at least 90% milestone timeliness.

2.2.1 Defect Prevention

Defect prevention focuses on preventing defects before they are introduced into the system instead of only finding them after implementation.

The team will apply the following practices:

- Clearly review requirements before implementation.
- Define acceptance criteria for important functions.
- Follow agreed coding standards.
- Perform code reviews before merging code.
- Use version control to manage source code changes.
- Perform testing during development.
- Record and track defects until they are resolved.
- Avoid implementing functions outside the agreed scope.

Defect prevention is particularly important because the project has a large number of Use Cases and a limited 13-week schedule.

2.2.2 Reviewing

Reviewing is the process of examining project requirements, design documents, source code, and other project outputs to identify problems before they become more expensive to fix.

The team will conduct:

- Requirements reviews
- Design reviews
- Code reviews
- Documentation reviews
- Sprint reviews

The project requires 100% of requirements and design documents to be reviewed, with no more than 5 identified defects during the reviewing stage.

2.2.3 Unit Testing

Unit testing verifies individual functions or modules independently.

Developers will create and execute unit test cases for core implemented functions. Unit testing will be performed during development so that defects can be detected early.

The project target is to test at least 80% of core implemented functions, with no more than 15 defects at this stage.

Examples of functions that should be covered by unit tests include:

- Login and authentication
- User management
- Risk score calculation
- Incident classification
- Notification functions
- Report generation
- AI alert processing

2.2.4 System Testing

System testing verifies the behavior of the integrated SecuraAI system. It focuses on whether different modules work correctly together and whether complete workflows satisfy the requirements.

The testing team will perform end-to-end scenarios covering important Use Cases and interactions between modules.

The target is to test at least 90% of implemented Use Cases, with no more than 8 defects.

Examples include:

- User login → authorization → access to functions
- Risk assessment → risk treatment → approval
- AI alert → incident confirmation → incident handling
- Policy creation → policy assignment → compliance assessment
- Incident management → related risk → post-incident report

2.2.5 Acceptance Testing

Acceptance testing verifies whether the final system satisfies critical business requirements and is ready for delivery.

The team will test 100% of critical business workflows. No critical defect should remain unresolved before final delivery.

The acceptance test target is no more than 3 defects.

2.2.6 Defect Management

All discovered defects will be recorded and tracked until resolution.

The basic defect management process is:

Defect Identified → Defect Recorded → Assigned to Developer → Fixed → Retested → Closed

Defects will be prioritized according to severity and impact. Critical defects affecting important security or business functions will receive the highest priority.

The team will also monitor defects continuously during development because the project plan identifies insufficient time for final integration, testing, and bug fixing as a project risk.

2.3 Training Plan

The training plan is prepared to ensure that team members have sufficient knowledge and skills to implement SecuraAI. Training will focus on technologies and technical areas that are important to the project.

Training Area	Participants	When, Duration	Waiver Criteria
NodeJs	Nguyen Van Hoai Le Thinh Le Dang Khoa Doan Thi Yen Nhi Nguyen Chi Duong	07/01/2026, 03 moths	Mandatory
NextJs	Nguyen Van Hoai Le Thinh Le Dang Khoa Doan Thi Yen Nhi Nguyen Chi Duong	11/05/2026, 01 moths	Mandatory
Docker	Nguyen Van Hoai Le Thinh Le Dang Khoa Doan Thi Yen Nhi Nguyen Chi Duong	01/08/2026, 01 moths	Mandatory

PostgreSQL	Nguyen Van Hoai Le Thinh Le Dang Khoa Doan Thi Yen Nhi Nguyen Chi Duong	08/08/2026, 01 moths	Mandatory
Subabase	Nguyen Van Hoai Le Thinh Le Dang Khoa Doan Thi Yen Nhi Nguyen Chi Duong	11/05/2026, 01 moths	Mandatory
Github	Nguyen Van Hoai Le Thinh Le Dang Khoa Doan Thi Yen Nhi Nguyen Chi Duong	23/05/2025, 03 months	Mandatory
System Analysis and Design	Nguyen Van Hoai Le Thinh Le Dang Khoa Doan Thi Yen Nhi Nguyen Chi Duong	23/05/2025, 03 months	Mandatory

3. Project Deliverables

The following table presents the main project deliverables of SecuraAI, including project reports, source code, database, and other project-related outputs. Each deliverable is planned and monitored according to the project schedule to ensure timely completion and successful project delivery.

#	Deliverable	Due Date	Notes
1	Report 1	dd/MM/yyyy	Completed project introduction document, including project overview, product background, existing systems analysis, business opportunity, software product vision, project scope, limitations, and references.
2	Report 2	dd/MM/yyyy	Project management plan, including scope and estimation, project objectives, risks, management approach, deliverables,

			responsibility assignments, communications, and configuration management.
3	Report 3	dd/MM/yyyy	
4	Report 4	dd/MM/yyyy	
5	Report 5	dd/MM/yyyy	
6	Report 6	dd/MM/yyyy	
7	Report 7	dd/MM/yyyy	
8	Source code of project SecuraAI	dd/MM/yyyy	Complete source code of the SecuraAI system, including frontend, backend, AI anomaly detection, database integration, and related system modules.
9	Database of SecuraAI	dd/MM/yyyy	Complete database schema and data of the SecuraAI system.

4. Responsibility Assignments

The following table describes the main responsibilities of each project member in completing the project deliverables defined in Section 3. The responsibilities are assigned based on each member's role, skills, and contribution to the project. D indicates the member who performs the task, R indicates the member who reviews the task, S indicates the member who provides support, and I indicates the member who is informed about the task. A blank cell indicates that the member is not directly involved in that responsibility.

D~Do; R~Review; S~Support; I~Informed; <blank>- Omitted

Responsibility	h o a i p v c e 1 8 1 7 4 4	t h i n h l c e 1 8 0 1 3 6	k h o a l d c e 1 8 1 0 3 0	n h i d t y c e 1 8 0 4 9 2	d u o n g n c c e 1 8 0 3 7 4
Report 1	D	D	D	D	D
1. Overview	D	R	S	R	D
2. Product Background	D	S	D	R	R
3. Existing Systems	R	S	D	D	R
4. Business Opportunity	S	R	D	D	S
5. Software Product Vision	R	R	S	D	D
6. Project Scope & Limitations	S	D	D	D	R
7. References	D	D	D	D	D
Report 2	D	D	D	D	D
1. Overview	S	D	R	R	S
2. Management Approach	R	S	S	D	R
3. Project Deliverables	S	S	R	R	D
4. Responsibility Assignments	R	R	S	S	D

Responsibility	h o a i p v c e 1 8 1 7 4 4	t h i n h l c e 1 8 0 1 3 6	k h o a l d c e 1 8 1 0 3 0	n h i d t y c e 1 8 0 4 9 2	d u o n g n c c e 1 8 0 3 7 4
Report 1	D	D	D	D	D
1. Overview	D	R	S	R	D
2. Product Background	D	S	D	R	R
5. Project Communications	R	S	D	S	R
6. Configuration Management	D	S	R	R	S
Report 3	D, S	D, R	D, R	D	D, R, S
Report 4	D	D, S	D	D, R	D, R, S
Report 5	D, S	D. R, S	D	D	D
Report 6	D, S, R	D	D	D	D, S

Responsibility	h o a i p v c e 1 8 1 7 4 4	t h i n h l c e 1 8 0 1 3 6	k h o a l d c e 1 8 1 0 3 0	n h i d t y c e 1 8 0 4 9 2	d u o n g n c c e 1 8 0 3 7 4
Report 1	D	D	D	D	D
1. Overview	D	R	S	R	D
2. Product Background	D	S	D	R	R
Report 7	D	D	D, S	D, S, R	D
Source code of project SecuraAI	D, R	D, R, S	D, R	S, R	D, S, R
Database of SecuraAI	D	D, R, S	D	D, R	D, S, R

5. Project Communications

Communication Item	Who/ Target	Purpose	When, Frequency	Type, Tool, Method(s)
Supervisor Meeting	Supervisor, Team Members	Review project progress, receive guidance, clarify requirements, discuss	FAP, 1 time/week	Microsoft Teams

		risks/issues and agree on next actions.		
Daily Team Coordination	All Team Members	Share current progress, planned work, blockers and coordinate tasks across modules.	Daily, 10–15 minutes	GitHub, Microsoft Teams
Technical & AI Integration Meeting	Back-end/Front-end Members	Discuss API contracts, log/data format, AI anomaly-detection integration, database changes and resolve technical dependencies.	Once per week or when integration issues arise	GitHub, Microsoft Teams /Merge Requests; API documentation
Code Review & Merge Discussion	Developer, Reviewer, Project Manager	Review source-code changes, verify coding quality and testing results, resolve conflicts and approve/reject merge requests.	For each merge request / before merging to main branch	GitHub Merge Request; comments; automated test results
Risk & Issue Communication	Project Manager, Responsible Member, Relevant Team Members	Report project risks/issues such as schedule delay, member unavailability, technical problems or integration failures; assign owner and response actions.	Immediately when identified; reviewed weekly	GitHub Issues; Microsoft Teams; risk/issue log
Report & Deliverable Review	Project Manager, Document Owner, Reviewer, Team Members	Review Report 2 and subsequent project documents/deliverables for completeness, consistency and submission readiness.	Before each deliverable deadline	Google Docs/Drive; Microsoft Teams; comments/review
Project Status Update	Project Manager All Team Members	Summarize completed work, remaining tasks, milestone status, important decisions and upcoming deadlines.	Once per week	Microsoft Teams / Zalo group / Telegram; GitHub Board

6. Configuration Management

6.1 Document Management

The team uses Google Drive as the primary tool for storing, sharing, and managing documents of the SecuraAI project.

Google Drive is a cloud-based storage service that allows users to store, access, and share documents over the Internet. It supports various file formats such as Microsoft Word, Microsoft Excel, and PDF, and integrates with Google Docs, Google Sheets, and Google Slides. Therefore, team members can access and work with project documents in a centralized location.

Project documents are stored in a shared folder on Google Drive. Team members are granted appropriate access permissions to view or edit documents according to their assigned responsibilities. The main documents managed include Project Weekly Reports, project reports from Report 1 to Report 7, system analysis and design documents, testing documents, meeting minutes, and other project-related documents.

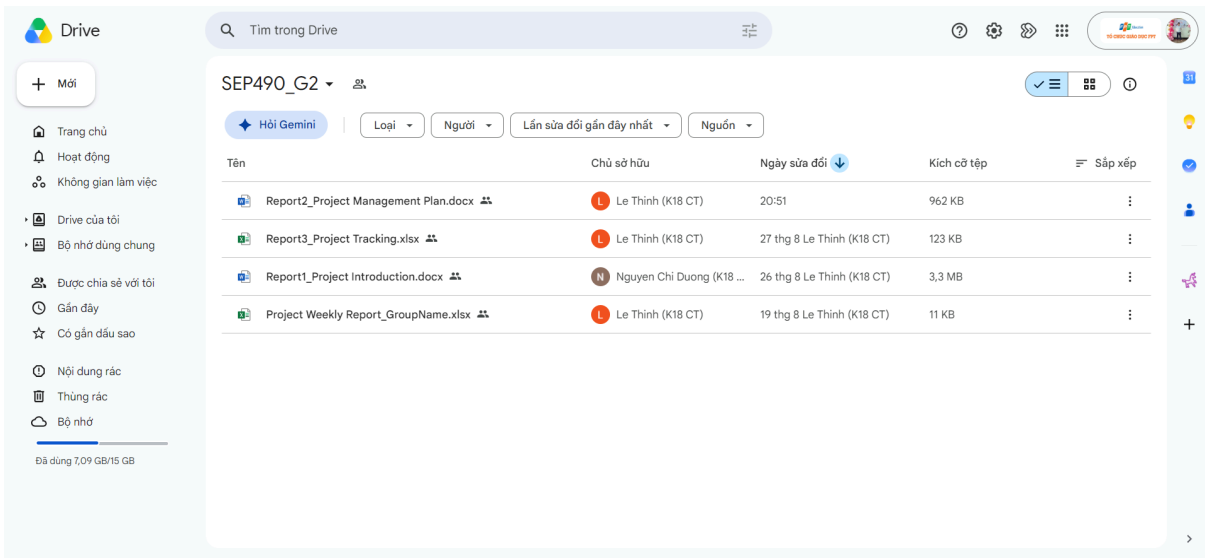


Figure 2: Document management of the SecuraAI project on Google Drive.

To manage document changes and versions, the team uses the Version Management feature provided by Google Drive. When a document is modified or updated, a new version can be stored on Google Drive while previous versions remain available for review or recovery when necessary.

For Microsoft Word or Microsoft Excel files uploaded to Google Drive, the responsible member can use the version management feature to upload a new version to replace the current one. Google Drive maintains information about file versions, including the update time and the person who made the changes. For important versions, the team can use the option to keep the version permanently to prevent it from being removed.

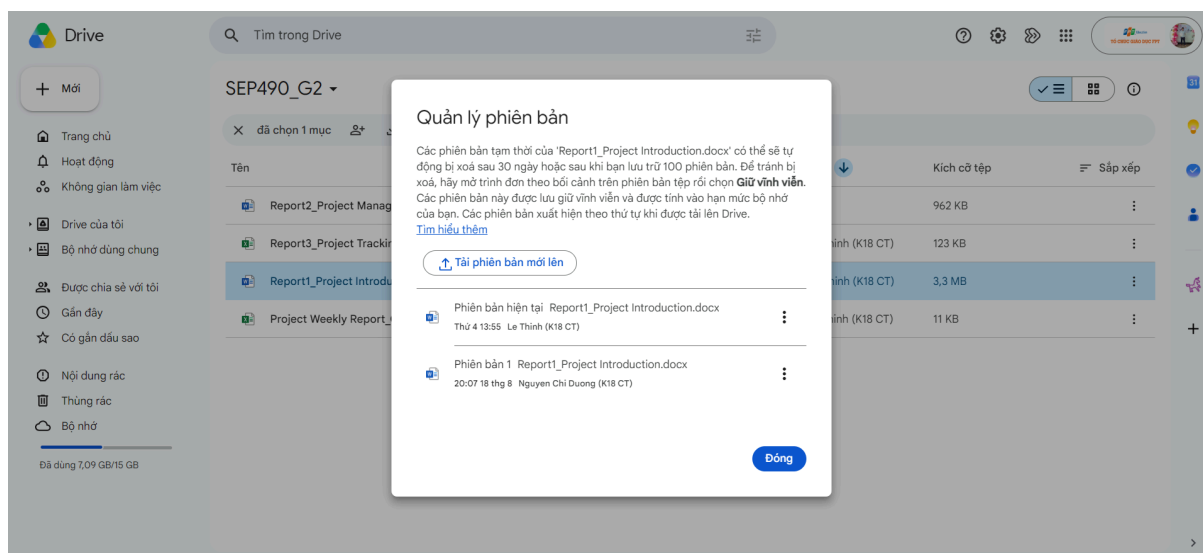


Figure 3. Version management of SecuraAI project documents on Google Drive.

The document management process is performed as follows:

1. The assigned member creates or updates the document according to the assigned task.
2. The document is stored in the appropriate project folder on Google Drive.
3. The member responsible for reviewing checks the content of the document.
4. If the document does not meet the requirements, the assigned member is requested to revise and update it.
5. The new version is uploaded to Google Drive, while the previous version is retained in the version history.
6. After the document has been reviewed and meets the requirements, that version is used as the official version of the project.
7. Important versions are retained so that they can be restored when necessary.

Advantages:

- Project documents are stored centrally in one location.
- Team members can access documents from different devices.
- Document sharing and access permission management are supported.
- Multiple team members can work on and update documents.
- Document version history can be maintained and managed.
- Previous versions can be restored when inappropriate changes are identified.
- The risk of document loss due to device failure is reduced.
- The tool is easy to use and suitable for team collaboration.

Disadvantages:

- A stable Internet connection is required to use all features effectively.
- Incorrect sharing permissions may allow unintended users to access project documents.
- When multiple members edit documents simultaneously, content management may become difficult without clear responsibility assignments.

- Version management for uploaded Microsoft Office files is less convenient than for documents created directly with Google Docs or Google Sheets.

Reason for Selection:

The team selected Google Drive because it provides the necessary features for project document management, including centralized storage, document sharing, access permission management, and version management. Google Drive is also easy to use and allows team members to access project documents from different devices without requiring the team to maintain a separate storage system.

In addition, the team currently uses Google Drive to store project documents such as the Project Weekly Report, Report 1, Report 2,.. and Report 7. Therefore, using Google Drive provides a consistent document management approach throughout the SecuraAI project.

6.2 Source Code Management

The team uses GitHub as the primary platform for storing, managing, and tracking source code changes throughout the development of the SecuraAI project.

GitHub is an online source code management and hosting platform that provides features for collaborative software development, such as Repository, Branch, Commit, Pull Request, Code Review, and access control. Through GitHub, team members can work on different functions, track the history of source code changes, and control changes before they are integrated into the main version of the system.

The source code of the SecuraAI project is divided into two separate repositories on GitHub:

- **SecuraAI-Frontend:** Used to manage the Frontend source code of the system, developed with Next.js.
- **SecuraAI-Backend:** Used to manage the Backend source code of the system, developed with Node.js and connected to the PostgreSQL database.

Separating the Frontend and Backend into two repositories allows the team to manage the two components independently. Team members can focus on the repository corresponding to their assigned responsibilities, while the change history of the Frontend and Backend can also be tracked separately and clearly.

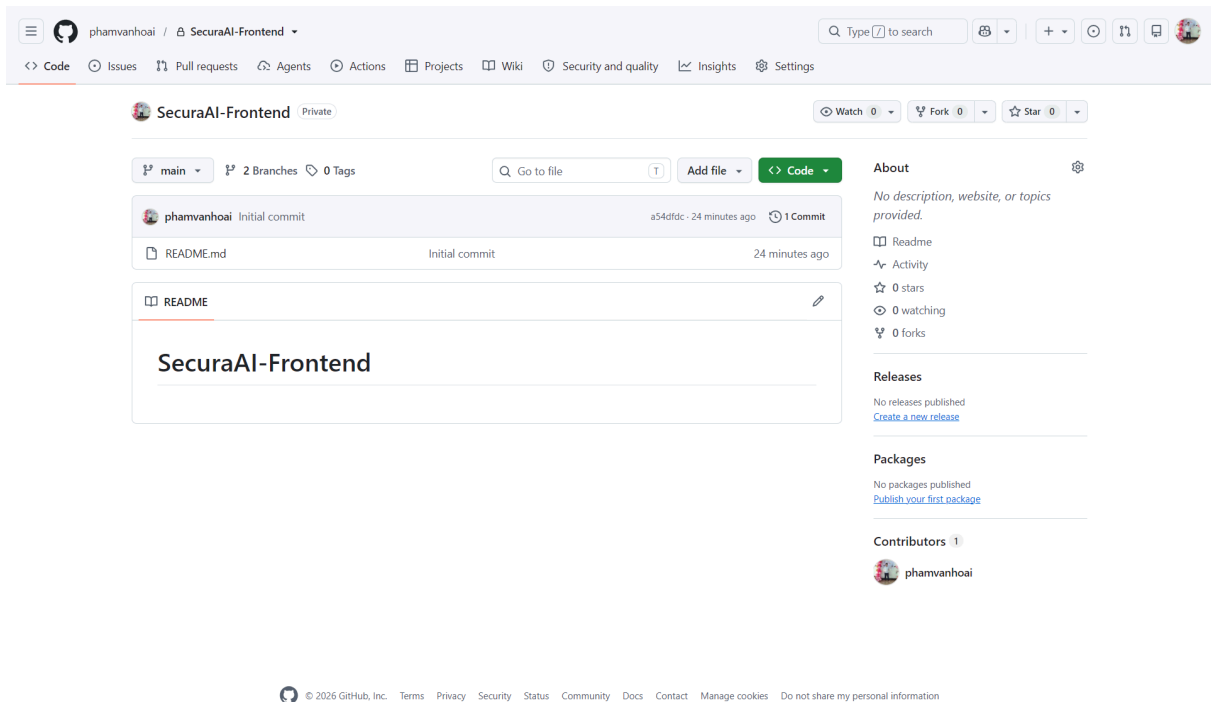


Figure 4. Frontend source code repository of the SecuraAI project on GitHub.

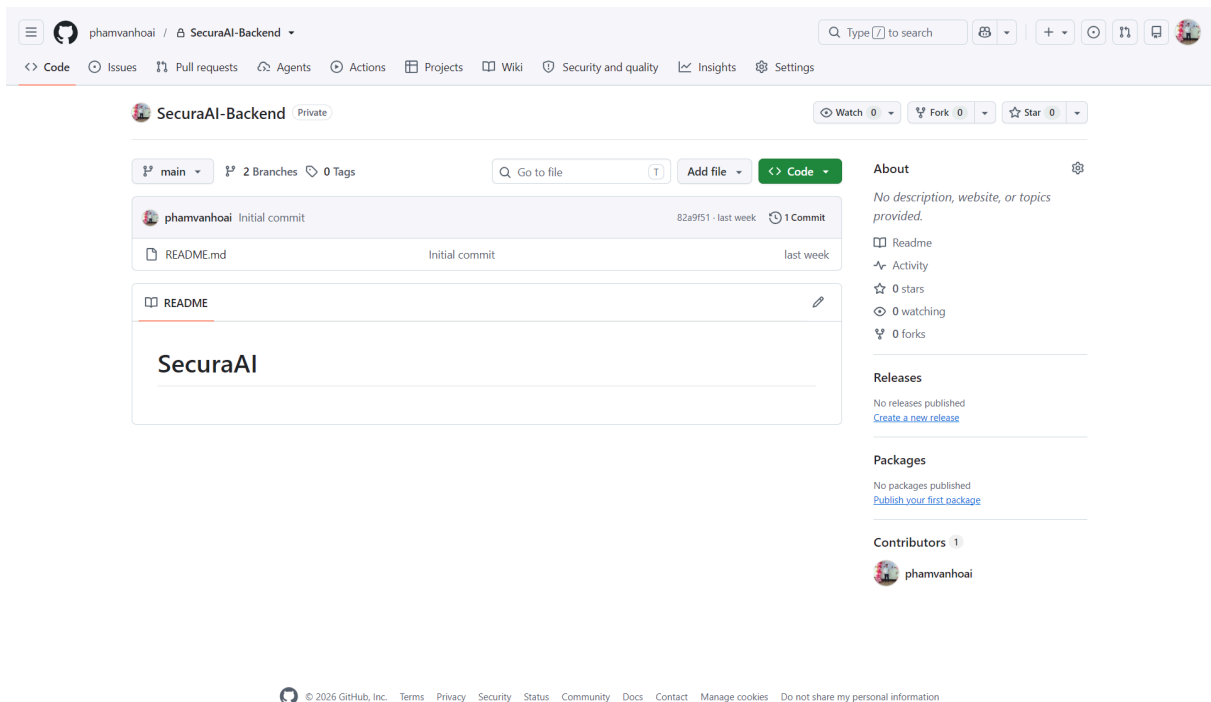


Figure 5: Backend source code repository of the SecuraAI project on GitHub.

In each repository, the team uses different branches to manage source code during development. The branches are managed as follows:

- **main:** Contains the stable version of the source code. Source code is integrated into this branch only after it has been reviewed and tested.
- **develop:** Contains source code being integrated during the development process. Completed functions are reviewed before being integrated into this branch.
- **feature branch:** Created separately for each function or assigned development task.
- **fix branch:** Used to resolve defects identified during development and testing.

Feature branches are named according to the function being implemented, such as feature/login, feature/user-management, or feature/risk-assessment. Fix branches are named according to the issue being resolved, such as fix/login-validation.

The source code management process is performed as follows:

1. When a member is assigned a function, the member obtains the latest source code from the develop branch of the corresponding repository.
2. The member creates a separate feature branch for the assigned function.
3. The member implements and tests the function on the feature branch.
4. During development, changes are committed regularly with clear descriptions and updated on GitHub.
5. After completing the function, the member creates a Pull Request to request integration of the feature branch into the develop branch.
6. The responsible reviewer checks the implemented function, source code quality, and compliance with the defined requirements.
7. When automated testing is configured, the related Unit Tests will be executed automatically before the Pull Request is accepted.
8. If the source code or test results do not meet the requirements, the Pull Request will not be accepted, and the responsible member must correct the identified issues.
9. When the source code meets the requirements and passes the required checks, the Pull Request is approved and integrated into the develop branch.
10. The source code on the develop branch is further integrated and tested with other functions of the system.
11. When the version on the develop branch is stable and satisfies the requirements of the Sprint or release, it is reviewed before being integrated into the main branch.

Advantages:

- Frontend and Backend source code can be stored and managed centrally on GitHub.
- Source code change history can be tracked throughout the development process.
- Multiple team members can work on different functions simultaneously.
- Branches help isolate source code changes for individual functions.
- Pull Requests support source code review before integration.
- Commits and Pull Requests help track the contributions of individual team members.
- Previous versions of the source code can be restored when necessary.
- Automated testing processes can be integrated into the source code management workflow.
- Separating the Frontend and Backend into two repositories provides clear and independent management of each component.

Disadvantages:

- Source code conflicts may occur when multiple members modify the same part of the system.
- Team members need to be familiar with Repository, Branch, Commit, and Pull Request features on GitHub.
- Improper branch management may cause difficulties during source code integration.
- Pull Request and Code Review processes require additional time before changes can be integrated.
- Using separate repositories for Frontend and Backend requires good coordination when changes affect both components.

Reason for Selection:

The team selected GitHub because it provides the necessary features for storing, managing versions, and coordinating source code development in a team-based software project. Features such as Repository, Branch, Commit, Pull Request, and Code Review allow the team to control source code changes before they are integrated into the main version.

GitHub is also suitable for the Agile Scrum approach applied in the SecuraAI project. During each Sprint, team members can develop assigned functions on separate branches and then review, test, and integrate the completed functions. This supports the continuous development, integration, and testing process defined in the project management plan.

6.3 Tools & Infrastructures

Category	Tools / Infrastructure
Technology	Next.js (FrontEnd), Node.js (BackEnd), Docker
Database	PostgreSQL
IDEs/Editors	Visual Studio Code
Diagramming	StarUML, DrawIO
Documentation	Microsoft Office, Google Docs/Sheets, Google Drive
Version Control	GitHub (Source Codes), Google Drive (Documents)
Deployment server	Localhost
Project management	TaskFlow , Google Sheets (Project Tracking)